

assume they take 1-bit per dimension (non-reduced dimension) to store. The input to the softmax layer is not sampled or projected.

In both cases, the average pool and bias gradients does not require saving since the gradient is constant.

For MNIST fully connected, this gives (per mini-batch element memory):

Baseline: $(784 + 300 + 300 + 300 + 10) \cdot 32 \text{ bits} = 6.776 \text{ kBytes}$

RAD 0.1: $(79 + 30 + 30 + 30 + 10) \cdot 32 \text{ bits} + (300 + 300 + 300) \cdot 1 \text{ bits} = 828.5 \text{ bytes}$

which leads to approximately 8x savings per mini-batch element.

For CIFAR-10 convolutional, this gives (per mini-batch element memory):

Baseline: $(3 \cdot 32 \cdot 32 + 16 \cdot 32 \cdot 32 + 32 \cdot 16 \cdot 16 + 32 \cdot 16 \cdot 16 + 32 \cdot 8 \cdot 8 + 10) \cdot 32 \text{ bits} = 151.59 \text{ kBytes}$

RAD 0.1: $(308 + 1639 + 820 + 820 + 205 + 10) \cdot 32 \text{ bits} + (16384 + 8192 + 8192 + 2048) \cdot 1 \text{ bits} = 19.56 \text{ kBytes}$

which leads to approximately 7.5x savings per mini-batch element.

For Sequential-MNIST RNN, this gives (per mini-batch element memory):

Baseline: $(784 \cdot (1 + 100) + 100 + 10) \cdot 32 \text{ bits} = 317.176 \text{ kBytes}$

RAD 0.1: $(784 \cdot (1 + 10) + 10 + 10) \cdot 32 \text{ bits} + (784 \cdot 100) \cdot 1 \text{ bits} = 44.376 \text{ kBytes}$

which leads to approximately 7.15x savings per mini-batch element.

FEEDFORWARD NETWORK TRAINING DETAILS

We trained the CIFAR-10 models for 100,000 gradient descent iterations with a fixed mini-batch size, sampled with replacement from the training set. We lower the learning rate by 0.6 every 10,000 iterations. We train with the Adam optimizer. We center the images but do not use data augmentation. The MNIST models were trained similarly, but for 20,000 iterations, with the learning rate lowered by 0.6 every 2,000 iterations. We fixed these hyperparameters in the beginning and did not modify them.

We tune the initial learning rate and ℓ_2 weight decay parameters for each experiment reported in the main text for the feedforward networks. For each experiment (project, same sample, different sample, baseline, reduced batch), for both architectures, we generate 20 (weight decay, learning rate) pairs, where each weight decay is from the loguniform distribution over 0.0000001 – 0.001 and learning rate from loguniform distribution over 0.00001 – 0.01.

We then randomly hold out a validation dataset of size 5000 from the CIFAR-10 and MNIST training sets and train each pair on the reduced training dataset and evaluate on the validation set. For each experiment, we select the hyperparameters that give the highest test accuracy.

For each experiment, we train each experiment with the best hyperparameters 5 times on separate bootstrapped resamplings of the full training dataset (50,000 for CIFAR-10 and 60,000 for MNIST), and evaluate on the test dataset (10,000 for both). This is to make sure the differences we observe across experiments are not due to variability in training. In the main text we show 3 randomly selected training curves for each experiment. Below we show all 5.

All experiments were run on a single NVIDIA K80 or V100 GPU. Training times were reported on a V100.

RNN TRAINING DETAILS

All RNN experiments were trained for 200,000 iterations (mini-batch updates) with a fixed mini-batch size, sampled with replacement from the training set. We used the full MNIST training set of 60,000 images whereby the images were centered. Three repetitions of the same experiment were performed with different seeds. Hyperparameter tuning was not performed due to time constraints.

The hidden-to-hidden matrix (W_{hh}) is initialised with the identity matrix, the input-to-hidden matrix (W_{ih}) and hidden-to-output (last hidden layer to softmax input) are initialised with a random matrix where each element is drawn independently from a $\mathcal{N}(0, 0.001)$ distribution and the biases (b_{ih} , b_{hh}) are initialised with zero.

The model was evaluated on the test set of 10,000 images every 400 iterations and on the entire training set every 4000 iterations.

For the "sample", "different sample", "project" and "different project" experiments different activations/random matrices were sampled at every time-step of the unrolled RNN.

All experiments were run on a single NVIDIA K80 or V100 GPU.

The average running times for each experiment are given in Table 2. Note that we did not optimise our implementation for speed and so these running times can be reduced significantly.

Table 2: Average running times for RNN experiments on Sequential-MNIST.

Experiment	Running Time (hrs)	GPU
Baseline	34.0	V100
Small batch	16.0	V100
Sample	96.0	K80
Different Sample	110.0	K80
Project	82.0	K80
Different Project	89.0	K80

IMPLEMENTATION

The code is provided on GitHub¹. Note that we did not optimize the code for computational efficiency; we only implemented our method as to demonstrate the effect it has on the number of gradient steps to train. Similarly, we did not implement all of the memory optimizations that we account for in our memory calculations; in particular in our implementation we did not take advantage of storing ReLU derivatives with 1-bit or the fact that average pooling has a constant derivative. Although these would have to be implemented in a practical use-case, they are not necessary in this proof of concept.

¹<https://github.com/PrincetonLIPS/RandomizedAutomaticDifferentiation>

FULL TRAINING/TEST CURVES FOR MNIST AND CIFAR-10

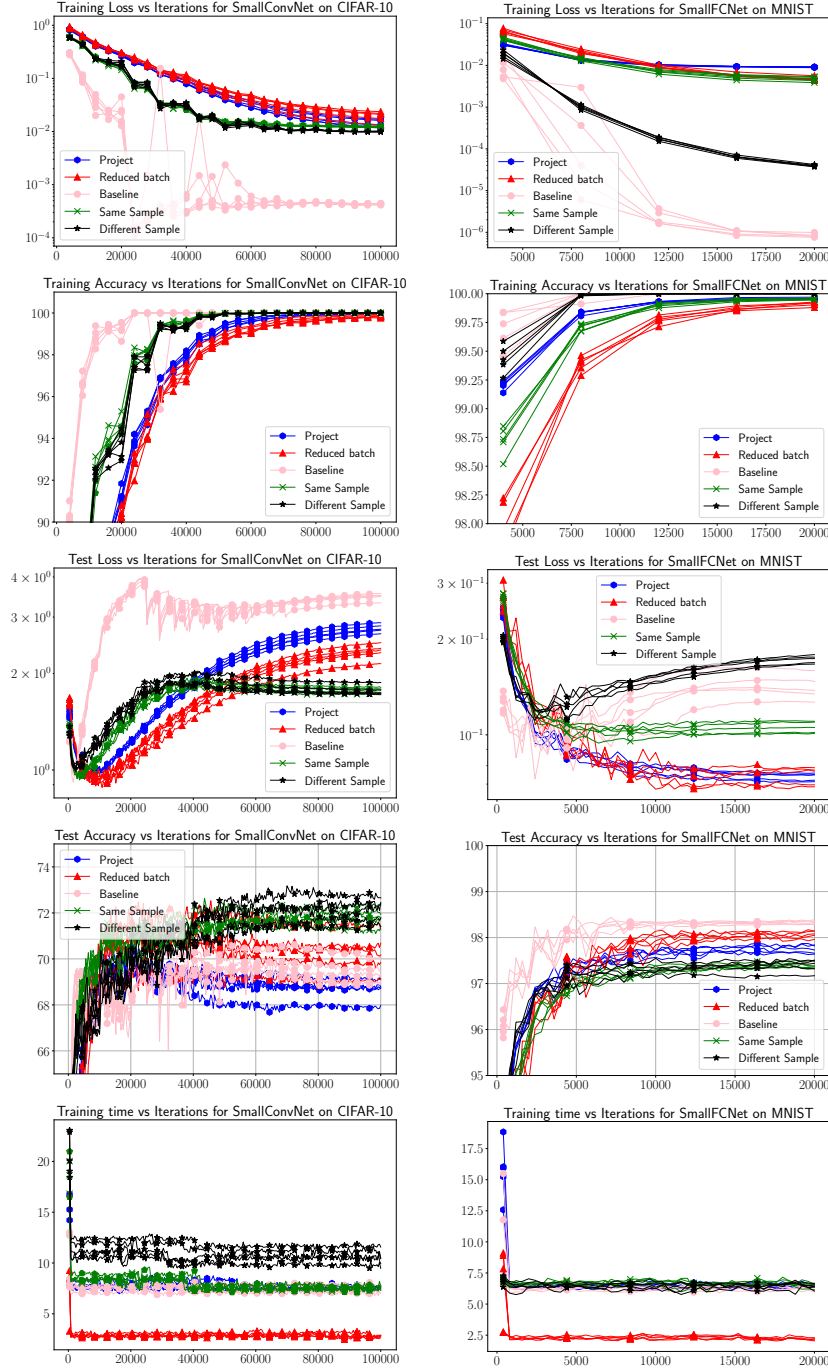


Figure 8: Full train/test curves and runtime per iteration. Note that training time is heavily dependent on implementation, which we did not optimize. In terms of FLOPs, “project” should be significantly higher than all the others and “reduced batch” should be significantly smaller. The “baseline”, “same sample”, and “different sample” should theoretically have approximately the same number of FLOPs.